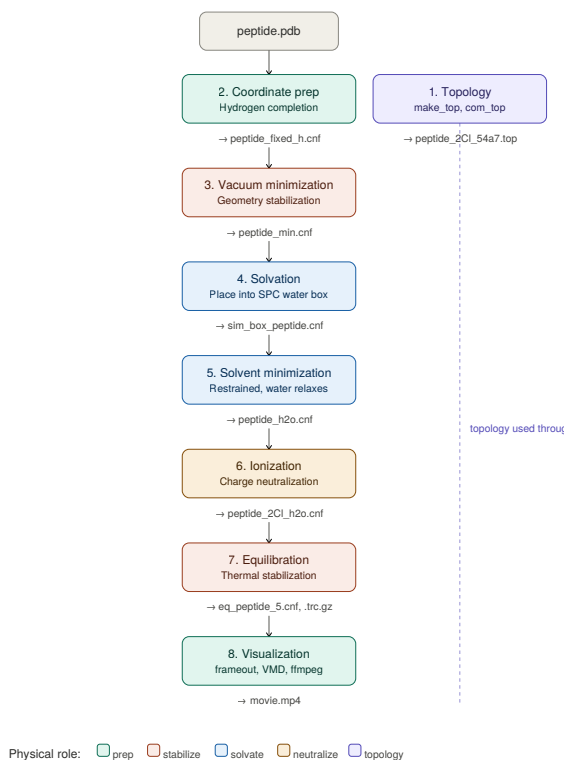


GROMOS peptide MD workflow

A step-by-step guide to building, solvating, equilibrating, and visualizing a peptide in a periodic box of SPC water using the GROMOS molecular simulation package with the 54a7 force field. Each section below shows the input files (left), the program invoked (center), the output files (right), and an example command line.

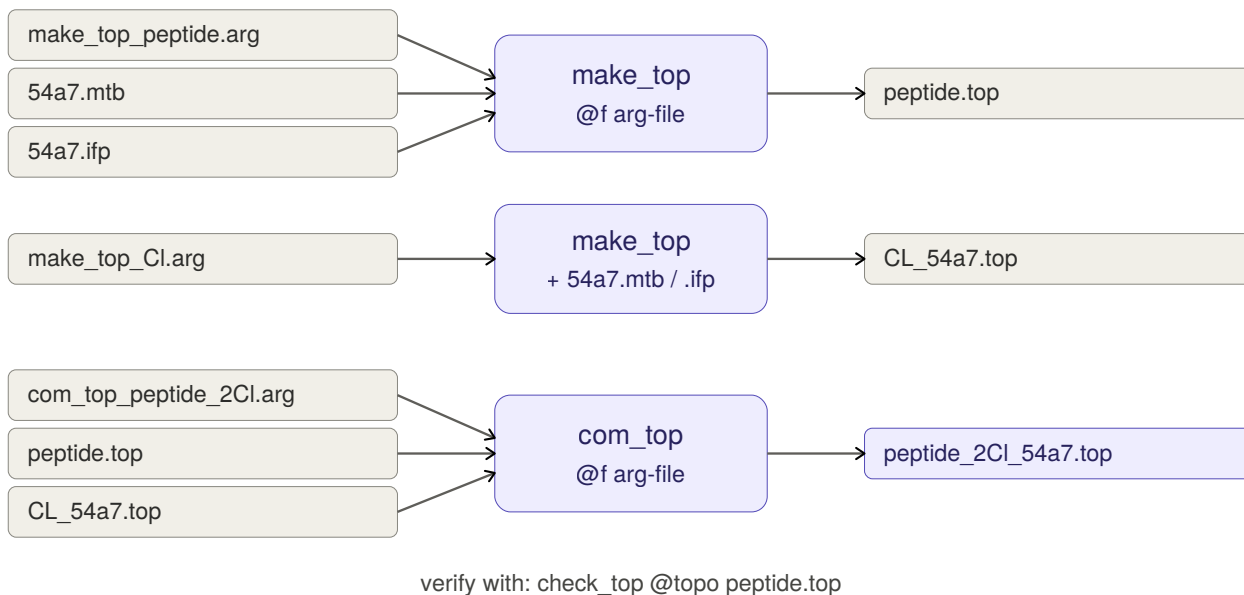
Workflow overview

The pipeline is an eight-stage chain. The topology is built once and reused throughout. Coordinates flow forward from one `.cnf` file to the next, with each stage progressively stabilizing the system: vacuum geometry first, then solvent, then charge, then temperature.



Step 1 — Topology generation

Three sub-commands run in this stage. `make_top` builds the peptide topology from the GROMOS 54a7 force field files (`54a7.mtb`, `54a7.ifp`); a second `make_top` call builds the chloride topology; `com_top` then combines them into the final system topology.



EXAMPLE COMMAND

```
# build peptide topology
make_top @f make_top_peptide.arg > peptide.top

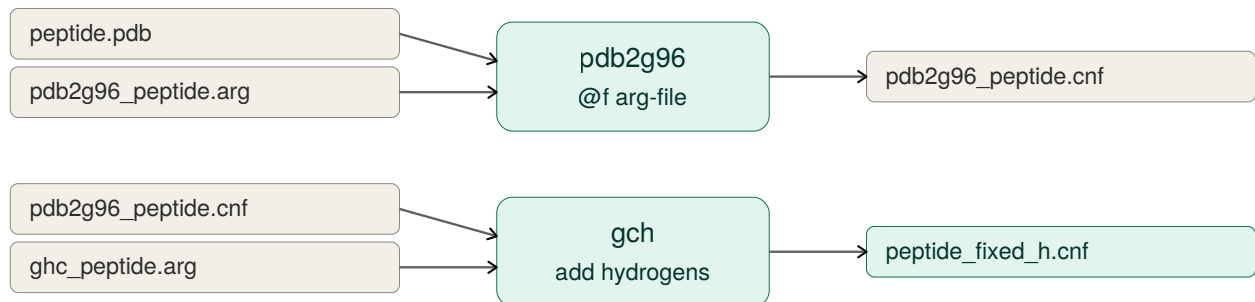
# build chloride topology
make_top @f make_top_Cl.arg > CL_54a7.top

# combine into the system topology
com_top @f com_top_peptide_2Cl.arg > peptide_2Cl_54a7.top

# sanity check
check_top @topo peptide_2Cl_54a7.top
```

Step 2 — Coordinate preparation

Two consecutive commands. `pdb2g96` converts the PDB into GROMOS coordinate format, then `gch` adds the missing hydrogen atoms (HYDROGEN_COMPLETION).



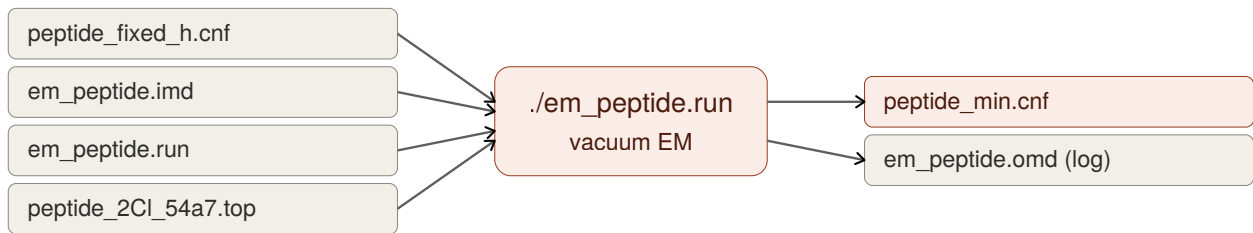
EXAMPLE COMMAND

```
# convert PDB → GROMOS .cnf
pdb2g96 @f pdb2g96_peptide.arg > pdb2g96_peptide.cnf

# add missing hydrogens
gch @f ghc_peptide.arg > peptide_fixed_h.cnf
```

Step 3 — Vacuum minimization

The first stabilization step. The shell script `em_peptide.run` invokes the GROMOS engine with the parameter file `em_peptide.imd`, the topology, and the hydrogen-completed coordinates.



EXAMPLE COMMAND

```
# make sure the run script is executable
chmod +x em_peptide.run

# launch vacuum energy minimization
./em_peptide.run

# inspect the log when it finishes
less em_peptide.ond
```

Step 4 — Solvation

`sim_box` places the minimized peptide into a periodic box pre-filled with SPC water (`spc.cnf`). The peptide is centered and the box dimensions are set from the arguments file.

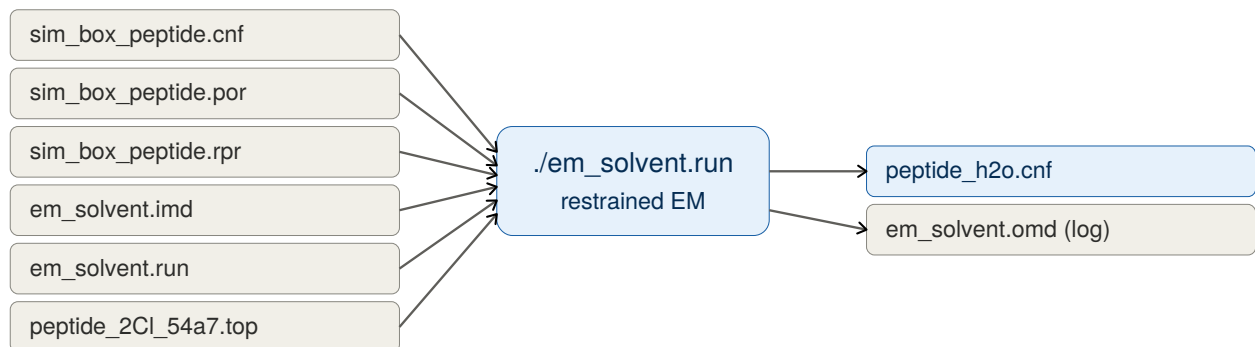


EXAMPLE COMMAND

```
# wrap the peptide in a box of SPC water
sim_box @f sim_box_peptide.arg > sim_box_peptide.cnf
```

Step 5 — Solvent minimization with restraints

The peptide is held in place by position restraints (`.por` selects the solute atoms; `.rpr` provides the reference coordinates) while the surrounding water is allowed to relax (SOLVENT_STABILIZATION).



EXAMPLE COMMAND

```
# make sure the run script is executable
chmod +x em_solvent.run

# launch restrained energy minimization
./em_solvent.run

# inspect the log
less em_solvent.omd
```

Step 6 — Ionization

The `ion` program replaces two water molecules with chloride ions to bring the system to zero net charge (CHARGE_NEUTRALIZATION).

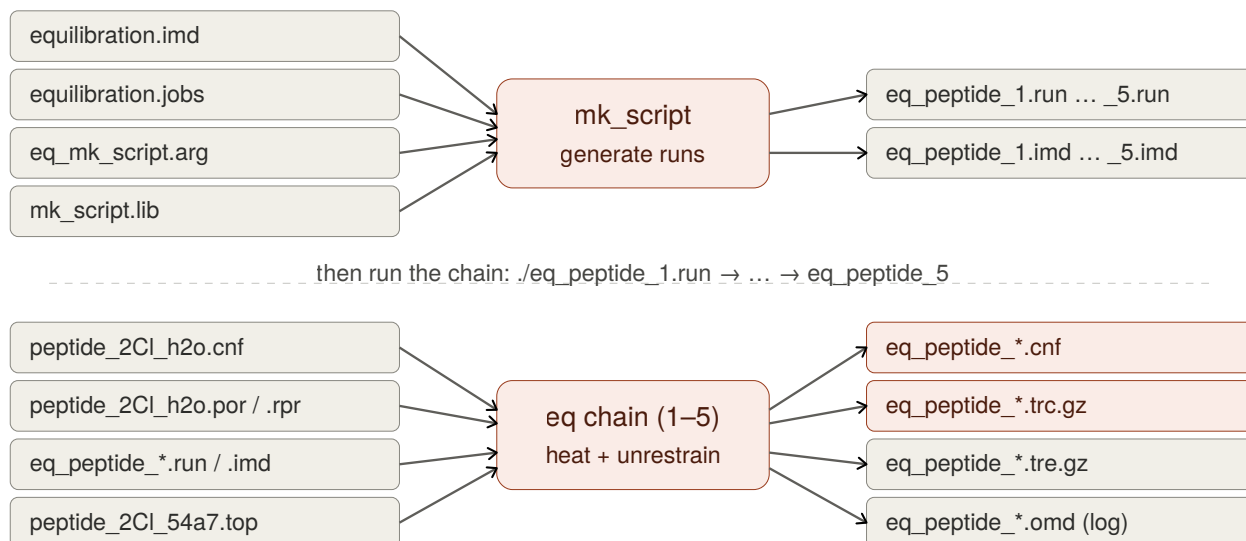


EXAMPLE COMMAND

```
# replace 2 waters with Cl- ions
ion @f ion_peptide.arg > peptide_2Cl_h2o.cnf
```

Step 7 — Equilibration

This stage runs as a chain of five sub-jobs (`eq_peptide_1` through `eq_peptide_5`) generated by `mk_script` from the template `equilibration.imd` and the schedule in `equilibration.jobs`. Each sub-job feeds its `.cnf` to the next, gradually heating the system to 300 K and reducing restraints (THERMAL_STABILIZATION).



EXAMPLE COMMAND

```
# generate the 5 equilibration scripts
mk_script @f eq_mk_script.arg

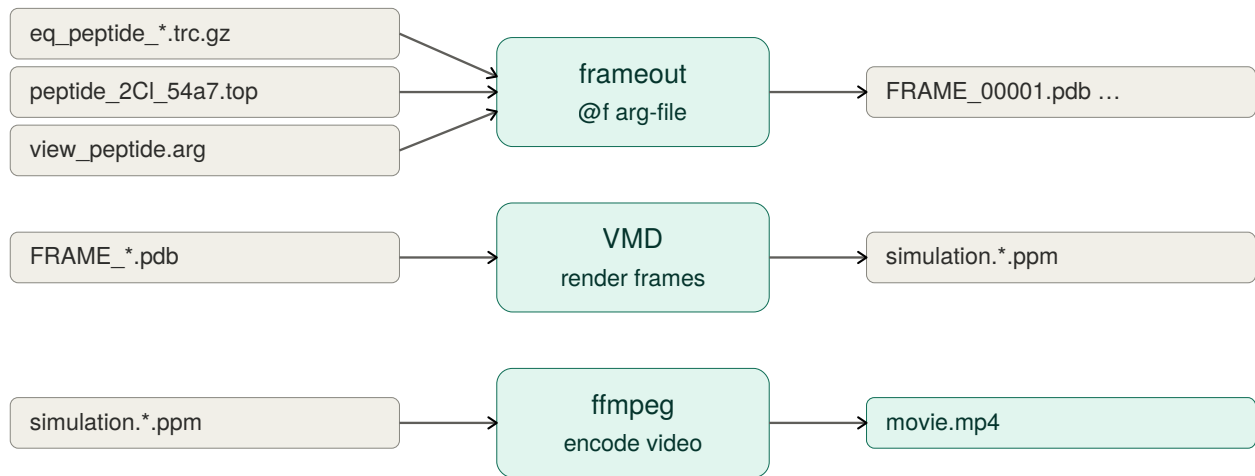
# make them executable
chmod +x eq_peptide_*.run

# launch the chain (each script queues the next one)
./eq_peptide_1.run

# the final structure ready for production MD
ls eq_peptide_5.cnf
```


Step 8 — Visualization

Three external programs in sequence. `frameout` extracts PDB frames from the trajectory, VMD renders each frame to a PPM image, and `ffmpeg` stitches the image sequence into a movie.



EXAMPLE COMMAND

```
# extract PDB frames from the equilibration trajectory
frameout @f view_peptide.arg

# render each frame with VMD (using a Tcl script that issues 'render Snapshot')
vmd -dispdev text -e render.tcl FRAME_*.pdb

# encode the PPM sequence into an mp4
ffmpeg -framerate 25 -i simulation.%05d.ppm -c:v libx264 -pix_fmt yuv420p movie.mp4
```